activity-logs-wiki-template の使い方

チャットで伝えるだけで日報・工数・週報・wiki がたまるリポジトリ

自分用のリポジトリを作って設定を確かめ、毎日の記録から週報と wiki につなげる

01 準備 必要なソフトウェアと開発者モードを整えてから自分用のリポジトリを作り、Claude Code と Copilot の設定を確かめる → P.2

02 スキルと wiki 同梱の Agent Skill の置き場所と使い分けを知り、wiki への取り込みを試す → P.10

03 毎日の使い方 業務開始・タスクの記録・業務終了を伝え、工数と週報を自動で作る → P.17

04 更新と保守 スキルの自動更新とテンプレートの取り込みを設定する → P.26

付録 チャットで使う言い方と、資料のデザインシステムを確かめる → P.29

準備

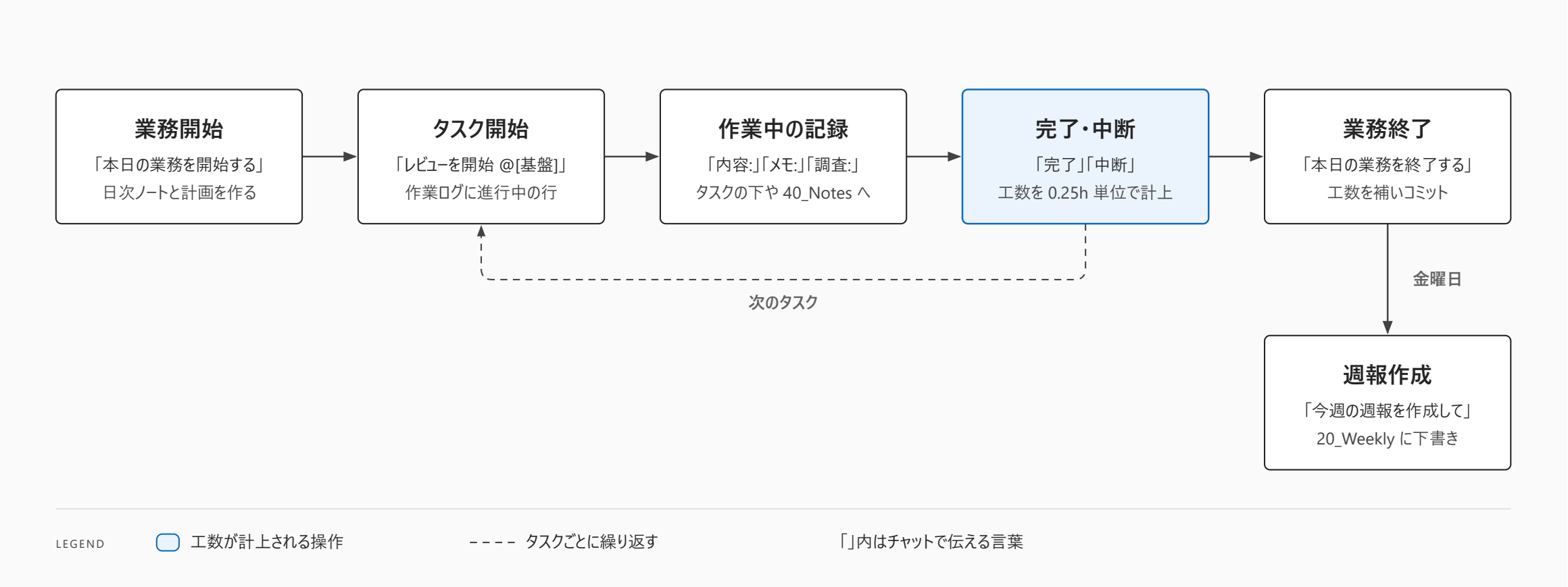
01 準備 P.2

02 スキルと wiki P.10

03 毎日の使い方 P.17

04 更新と保守 P.26

チャットで一言伝えるだけで、日次ノート・工数・週報・wiki がたまっていく



ファイルの作成と編集はエージェントが AGENTS.md と procedures/ の手順に従って行い、push だけは利用者が行う

Node.js 22 以上と uv、headless shell まで入れておけば、記録から資料作りまで動く

ソフトウェア	用途
Windows 11・PowerShell	シェルに使い、secret-scan フックは PowerShell 5.1 で動く（bash は Git 同梱のものに限る）
Git	業務終了時のコミットと、サブモジュールの初期化に使う
Claude Code か Copilot	エージェント本体として、AGENTS.md と procedures/ の手順で記録を書く
Node.js 22 以上（npm・npx）	Playwright MCP と foam MCP を npx で起動し、スライドの PPTX 変換も 22 以上を求める
Python	wiki の点検（check_evidence.py）と、図・スライドのスクリプトを動かす
uv	natural-japanese の機械チェックを動かし、依存は実行時に自動で入れる
Microsoft Edge	Playwright MCP が --browser msedge で操作する
Chromium headless shell	スライドの PDF 化と検査に使い、clone 後に npm run setup で入れる（Edge では代用できない）
VS Code・Foam 拡張（推奨）	ノートの閲覧と、Alt+D での日次ノート作成に使う

出典: 40_Notes/2026-10-26_prerequisites-software.md

PPTX 変換や図の PNG 書き出しに要るものは、その作業をするときに足せばよい

作業・スキル	必要なもの	補足
スライドの PPTX 変換	python-pptx	動画を埋め込むときは ffmpeg も入れる
スライドの自己テスト	pdftoppm・ffmpeg	無いテストは飛ばされる
スライドの PR の作成	gh	利用者が了承したときだけ使う
図の PNG 書き出し	Python 版の playwright と Chromium	wiki の図は SVG なので、普段は要らない
design-taste-frontend	Chrome または Edge	CHROME_PATH で場所を指定できる
design-artifact	tot	利用者の了承を得てから入れる
natural-japanese の semantic.py	sentence-transformers	初回に約1GBのモデルを取得する
スキルの自動更新	手元には不要	GitHub Actions の上で Node.js 22 が動く

出典: 40_Notes/2026-10-26_prerequisites-software.md（Python 3.14 で sudachipy と Python 版 playwright が入るかは要確認）

clone の前に開発者モードと core.symlinks を有効にしないと、スキルのリンクが作られない

clone の前に行う設定

モードを有効化	Windows の「設定」、「システム」、「開発者向け」の順に開き、開発者モードをオンにする
設定を確かめる	`git config --global --get core.symlinks` で true と表示されるかを確かめる
無ければ設定する	true と表示されなければ `git config --global core.symlinks true` を実行する
リンクの確認	clone 後に `Get-ChildItem .claude/skills | Select-Object Name, LinkType` で SymbolicLink と表示される

設定せずに clone したとき

- Git がリンクを通常のテキストファイルとして取り出す
- Claude Code が .claude/skills/ からスキルを読めなくなる
- 設定を済ませてから clone し直すと、リンクとして取り出せる

テンプレートから自分用のリポジトリを作り、サブモジュールごと手元に取り出す

テンプレートから作る	ogis-dac-rd/activity-logs-wiki-template の「Use this template」から作り、公開範囲は Private にする
手元に取り出す	スライド作成スキルのサブモジュールごと `git clone --recurse-submodules <URL>` で取り出す
ファイル名を表示	`git config core.quotepath false` で、日本語のファイル名を git status で読めるようにする
個人設定を書く	エージェントが作業前に必ず読む profile.md に担当領域と既定の @[カテゴリ/プロジェクト名] を書き、共有しない設定は CLAUDE.local.md に書く
週報の名前を書く	profile.md の「自分の名前（週報に使用）」に書くと、週報の件名と本文の {自分の名前} に入る

Windows では clone の前に開発者モードを有効にする（→ P.5）

モデルとエフォートは settings.json で決まり、worktree を使わない設定だけはアプリで手動で行う

.claude/settings.json の設定値

キー	値	意味
`model`	`sonnet`	会話の本体で使うモデル
`advisorModel`	`opus`	作業の前後に意見を求めるアドバイザー
`effortLevel`	`medium`	推論の深さ
`CLAUDE_CODE_SUBAGENT_MODEL`	`haiku`	調べものを任せるサブエージェント
`CLAUDE_CODE_USE_POWERSHELL_TOOL`	`1`	コマンドを PowerShell で実行する
`enabledMcpjsonServers`	4件	確認なしで有効にする MCP サーバー

settings.json で設定できないもの

- デスクトップアプリはセッションごとに worktree を作ることがある
- 日次ノートは1つの作業ツリーに書き足すので、worktree は使わない
- 無効にする設定は、利用者がアプリ側で手動で行う
- 設定画面と項目名は要確認として、説明会で画面を見て確かめる

よく使う安全なコマンドは確認なしで通し、削除と push は設定とフックで禁止している

区分	対象	狙い
確認なしで許可	• scripts/ の実行、Get-ChildItem、Test-Path、New-Item -ItemType Directory、Move-Item • git の status・diff・log・add・commit・remote・fetch template・show・rev-parse と submodule の add・update・status • check_evidence.py と knowledge-graph-extend.py の実行、ナレッジグラフの中間ファイルの削除（rm -rf 90_Wiki/.ua/intermediate）	毎日の記録とコミットのたびに承認を求められないようにする
禁止	• Remove-Item・del などの削除系、git push、git reset --hard • Foam MCP の書き込み系ツールと、.env の読み取り	記録の消失と、GitHub 上の履歴の書き換えを防ぐ
フック	• Bash・PowerShell の前に command-guard.ps1 が削除系と push を拒否し、rm は上の中間ファイルの削除だけを通す • Write か Edit のたびに secret-scan.ps1 が機密らしき文字列を探し、マスクを促す	設定では書けない「この形だけ許す」を判定し、認証情報が残るのを防ぐ
プラグイン	understand-anything（ナレッジグラフ）を project スコープで有効にし、版をタグで固定する（版が上がったら、登録済みの PC は登録し直す）	全員が同じ版を使う

MCP サーバーは settings.json の enabledMcpjsonServers で有効になり、個人の設定は .claude/settings.local.json（.gitignore 済み）に書く

GitHub Copilot でも AGENTS.md を読み、削除と push はフックで拒否する

Copilot が読むファイル

ルール	AGENTS.md（Copilot CLI は CLAUDE.md も読む）
MCP サーバー	VS Code は .vscode/mcp.json、Copilot CLI は .mcp.json
スキル	.agents/skills/ と .claude/skills/
拒否・機密検査	.github/hooks/agent-hooks.json のフック
自動承認	VS Code は .vscode/settings.json、CLI は --allow-tool などの起動オプション

Claude Code との違い

- .claude/settings.json を読まないので、使う前のフック command-guard.ps1 が削除系、git push、git reset --hard、.env の読み取りを拒否する（Claude Code も同じフックを使う）
- 使った後のフックで secret-scan.ps1 を -Copilot 付きで動かし、機密らしき文字列のマスクを促す
- MCP の設定は書式の違う2ファイルにあり、サーバーを増減するときは両方を直す
- CLAUDE.md の「Claude Code 専用」の節と .claude/agents/ は使わない

スキルと wiki

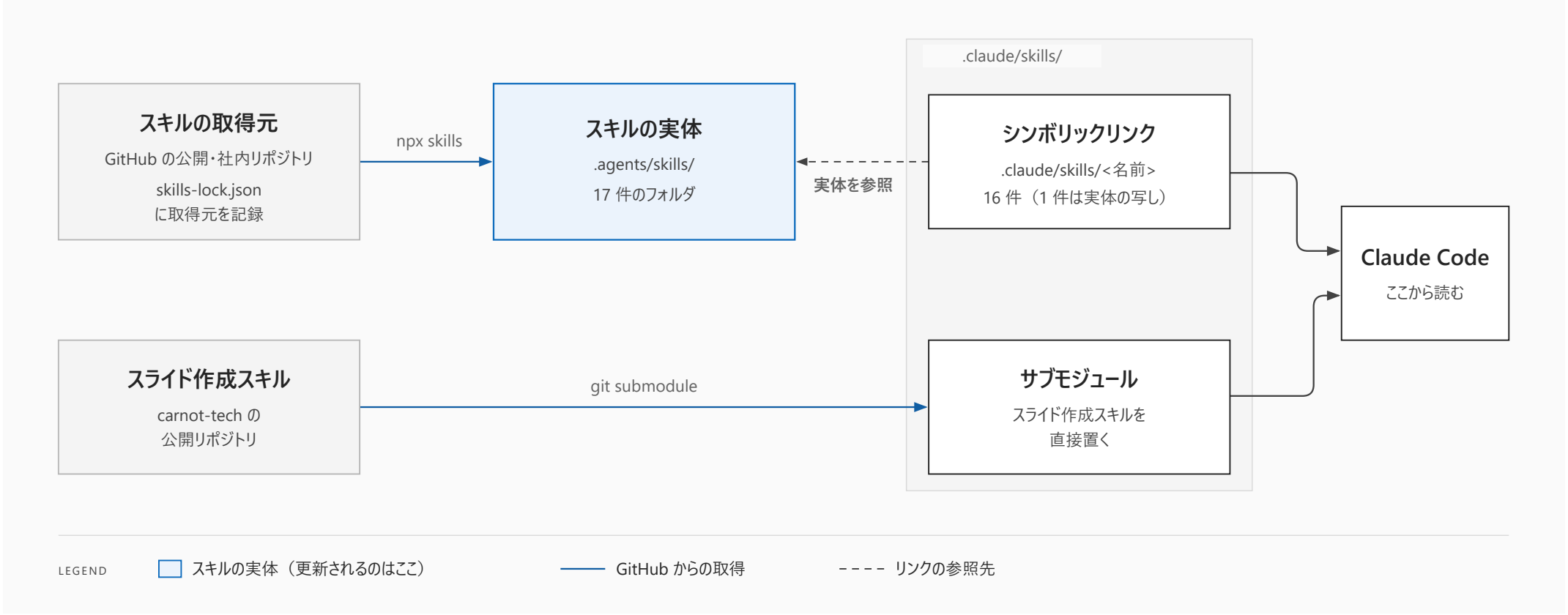
01 準備 P.2

02 スキルと wiki P.10

03 毎日の使い方 P.17

04 更新と保守 P.26

スキルの実体は .agents/skills/ に1つだけ置き、Claude Code はシンボリックリンク経由で読む



Windows では開発者モードが無いとリンクが作られない可能性がある（→ P.5）

記録と文章のスキルが書き方をそろえ、調査のスキルが選択肢の洗い出しから調べものを引き受ける

スキル	取得元	用途
llm-wiki	リポジトリに同梱	wiki の取り込み・問い合わせ・点検
foam-vscode	リポジトリに同梱	Foam の wikilink・テンプレート・日次ノートの書き方
git-conventions	`copilot-custom-hayashi`	コミットメッセージ・ブランチ名・PR の書き方
keep-steps-simple	`copilot-custom-hayashi`	応答を実行しやすい形に整える（profile.md の指定で自動、または /keep-steps-simple）
japanese-tech-writing	`copilot-custom-hayashi`	日本語の技術文書を書く・推敲する
natural-japanese	`coji/natural-japanese`	仕事の日本語文書を自然で読みやすく直す
tech-options-pickup	`copilot-custom-hayashi`	調査対象の選択肢を洗い出す
tech-research	`copilot-custom-hayashi`	選択肢ごとの特性・制約・リスクを調べる

copilot-custom-hayashi は ogis-dac-rd の社内リポジトリ

図と資料のスキルを使えば、構成図・HTML の資料・スライドまでリポジトリの中で作れる

スキル	取得元	用途
diagram-design	`cathrynlavery/diagram-design`	構成図やフロー図を SVG で描き、資料と wiki に入れる
design-taste-frontend	`Leonxlnx/taste-skill`	Web ページの見た目を決めて作り、ブラウザで検査する
html	`plannotator/effective-html`	1ファイルの HTML 資料を作り、専門のスキルに振り分ける
design-artifact	`plannotator/effective-html`	HTML の配色・書体・レイアウトを決める
html-diagram	`plannotator/effective-html`	関係や順序を見せる HTML の図を作る
html-plan	`plannotator/effective-html`	計画やロードマップを HTML で整理する
html-prototype	`plannotator/effective-html`	画面のモックアップや試作品を作る
html-wireframe	`plannotator/effective-html`	情報構造を確かめるワイヤーフレームを作る
big-pics-little-words	`plannotator/effective-html`	大きな図と少ない言葉で説明する（/big-pics-little-words で呼び出す）
スライド作成スキル	carnot-tech のサブモジュール	設計規約に沿って HTML のスライドを作る

技術調査は tech-options-pickup で候補を決めてから tech-research で個別に調べ、1回で使用量の約1割を使う（プランモードで使用した場合）

調査の進め方

候補を決める	「選択肢を洗い出したい」と頼むと、tech-options-pickup がスコープと候補の一覧を作る
個別に調べる	「各選択肢を調べたい」と頼むと、tech-research が候補ごとの特性・制約・リスクをレポートにする
情報源	Web ページを一次情報源として実際に取得し、分からない点は「不明」「要確認」と書く
扱わない範囲	候補どうしの優劣の比較と推奨案は作らないので、判断は利用者が行う
置き場所と図	レポートは 40_Notes/ に置き、図にできる箇所は説明付きの図にする

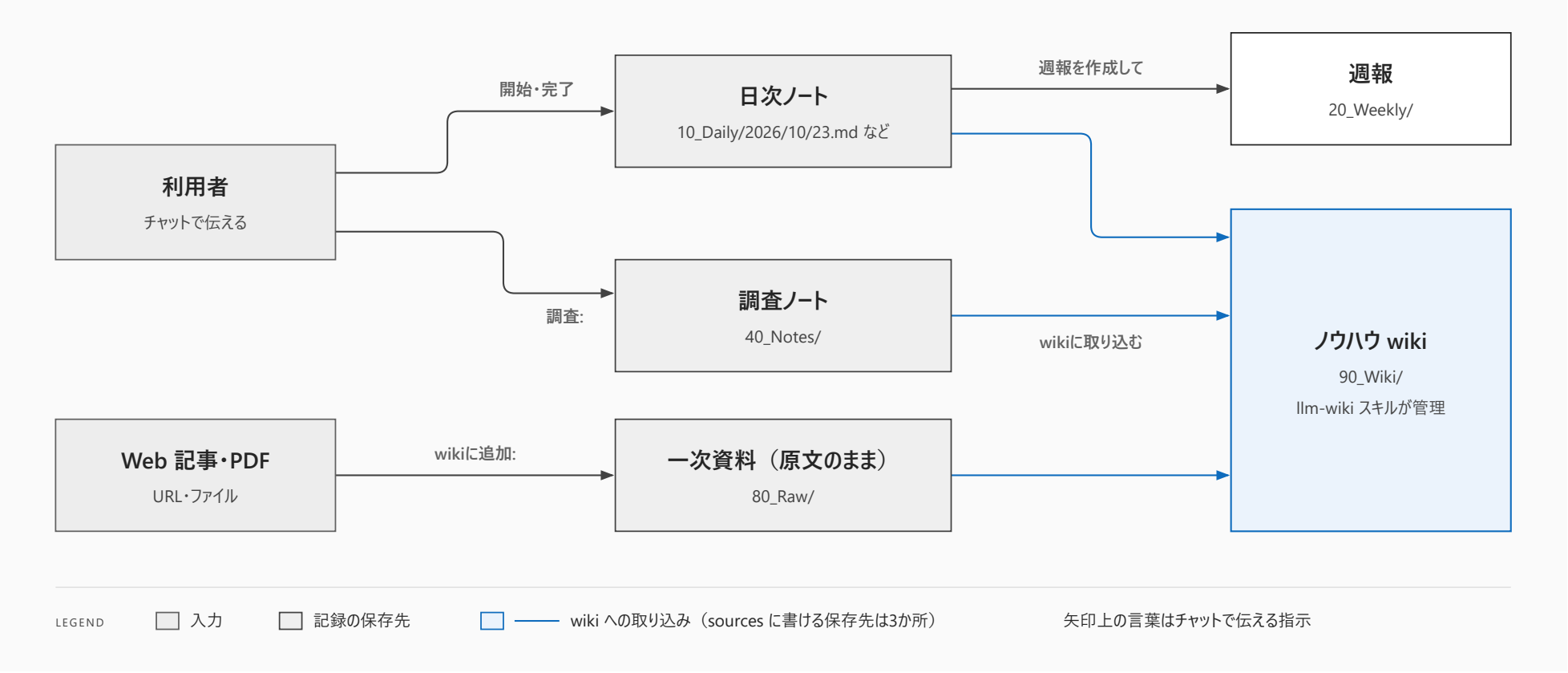
1回の調査で使う量

tech-research 1回あたり（作成者の実測、プランモードで使用した場合）

10〜12%

Claude の使用量上限に対する割合で、続けて回すときは残量に注意する

「wikiに追加: <URL>」の一言で、原文を 80_Raw/ に残してから 90_Wiki/ の記事にまとめる



wiki の記事の根拠に書けるのは 80_Raw/・10_Daily/・40_Notes/ の下のファイルだけ

wiki への取り込みでは図表の中身まで文字で残し、既存の記事と食い違えば両論を残す

「wikiに追加: <URL>」で行われること

取得して残す	ページを実際に取得し、80_Raw/ に原文のまま保存する
図表を読む	図表やスクリーンショットは、読み取った内容を「画像の読み取り」として元の位置に残す
振り分ける	既存の記事と照らし、新規・更新・見解相違・新規知見なしのどれかに分ける
記事にする	記事を書き、図にできる箇所は SVG か Mermaid の図にし、影響する他の記事と index.md・log.md も更新する

取り込んだ後の使い方

- 「wikiで調べて」と頼むと、wiki を検索して出典付きで答える
- 「wikiを点検」と頼むと、根拠とリンクのずれを確かめる
- 読んで確かめた記事は「確認した」と伝えると reviewed になる

「ナレッジグラフを更新して」で、wiki の記事と根拠のつながりをグラフにする

行われること（Claude Code のみ）

解析する	understand-anything プラグインの /understand-knowledge が 90_Wiki/ を読む
補う	knowledge-graph-extend.py が、プラグインの読めない部分をグラフに足す
保存する	90_Wiki/.ua/knowledge-graph.json に書く（Git の管理外）

手順は procedures/knowledge-graph.md（記事が1件以上あるときに使え、グラフのダッシュボードを開くには pnpm が要る）

リポジトリ側で補うもの

- 80_Raw/ の資料と、記事の sources に書いた根拠ファイル
- [[10_Daily/…]] など、wiki の外へのリンク
- [[90_Wiki/…]] の形の、記事どうしのリンク
- index.md の節によるカテゴリ

毎日の使い方

01 準備 P.2

02 スキルと wiki P.10

03 毎日の使い方 P.17

04 更新と保守 P.26

「本日の業務を開始する」の一言で、日次ノートの作成と前回の持ち越しの読み込みが終わる

利用者が伝えること

業務開始	「本日の業務を開始する」
計画の追加	「設計書の更新を追加して」
計画の取り消し	「定例会議は今日はやらない」
VS Code から	Alt+D（Foam: Open Today's Note）で同じテンプレートから作る

エージェントが行うこと

- 今日の日次ノートをテンプレートから作り、既にあれば再開する
- 前回の日次ノートの「次:」を今日へのリンクに書き換える
- 前回の持ち越しを、今日の「やること」の先頭に入れる
- 休日を挟んでも、今日より前で直近の日次ノートから持ち越す
- ファイルのパスと計画・持ち越しの件数を短く報告する

keep-steps-simple を有効にすると、応答の1行目が次にやることになり、手順には番号が付く

次の行動を先頭に	1行目に、読み手がすぐ実行できる行動が来る
手順に番号を振る	1つの手順には1つの行動だけが書かれる
現在地を言い直す	ターンをまたぐとき、どこまで進んだかが書かれる
所要時間を見積もる	「数分」か「数時間」かを区別できる形で示される
成果と次の一手	完了した成果が示され、最後に次の行動が1つ書かれる

profile.md に使うよう書けばエージェントが自分で有効にし（書いていなければ /keep-steps-simple と入力する）、「normal mode」と伝えると解除される

作業の区切りごとに一言伝えると、作業ログ・タスクの下・各セクションに書き分けられる

種別	言い方の例	記録される場所
開始	「API設計レビューを開始 @[クラウドデータ基盤]」	作業ログに進行中の行を追記する
内容	「内容: 命名規則を統一する方針にした」	タスクの下に書き、長い内容は 40_Notes/ に切り出す
完了	「完了。指摘は5件」	作業ログを完了に更新し、工数表に計上する
中断	「中断。明日続ける」	作業ログを中断に更新し、持ち越しにも入れる
メモ	「メモ: 会議室を予約する」	タスクの下か「週報に記載しないメモ」に書く
調査	「調査: Foam の wikilink の仕様」	40_Notes/ に調査ノートを作ってリンクする
問題	「問題: ビルドがタイムアウトする」	「問題」に詳細・原因候補・対応策・結果の欄を作る
アイデア	「アイデア: 工数表を週報に自動転記したい」	「アイデア」に見出しを立てて書く
学んだこと	「学んだこと: Foam は H1 を title に使う」	「学んだこと」に見出しを立てて書く

開始・内容・完了の発言だけで、作業ログと工数表の行がそろう

チャットで伝えたこと

```
API設計レビューを開始 @[クラウドデータ基盤]
内容：エンドポイントの命名規則を統一する方針にした
API設計レビューが完了
定例会議を開始 @[庶務]
定例会議が完了
```

日次ノートに書かれた内容

```
### やること（[x] = 完了）
- [x] API設計レビュー
  - [内容] エンドポイントの命名規則を統一する方針にした
- [x] 定例会議

## 📝 作業ログ
- 09:30 〜 11:00 [完了] @[クラウドデータ基盤] API設計レビュー
- 11:00 〜 12:00 [完了] @[庶務] 定例会議

## ⏱️ 工数
| @[クラウドデータ基盤] | A313271100 | API設計レビュー | 1.50 |
| @[庶務]               | A295481100 | 定例会議        | 1.00 |
| **合計**              |            |                 | 2.50 |
```

「本日の業務を終了する」で工数表を整えてからコミットし、push は利用者が確かめてから行う

エージェントが行うこと

コードを補う	工数表のプロジェクトコードの空欄を project-codes.md で埋める
合計を確かめる	「合計」行が各行の合計と合っているかを確かめる
未登録を伝える	対応表に無い @[...] があれば報告する
コミットする	`docs(daily): 2026-10-23 の作業ログを記録した` の形でコミットする

利用者が行うこと

- コミットの内容を確かめてから git push する
- 終わらなかったタスクは「中断」と伝えるか、持ち越しに入れておく
- 未登録と報告された @[...] は、対応表に追加すれば週報の作成時に補われる

工数は完了か中断を伝えた時点で 0.25h 単位に丸めて計上され、手で表を書く必要はない

計上のしかた

完了	開始からの所要時間を丸めて表に記入し、同じタスクがあれば加算する
中断	中断までの時間を加算し、再開後の分は完了時に足す
工数を指定	「工数は1.5h」と伝えれば、その値が優先される
修正	「定例会議の工数を1hに直して」と伝えれば直る
コード	project-codes.md の対応表から引き、無ければ空欄で報告する

プロジェクトコードの追加

```
プロジェクトコードを追加:
TECH:研究開発　A295483100　ー　社内
【スキル】技術研鑽　【スキル】技術研鑽
```

- 工数システムの Excel の行（タブ区切りの6列）をそのまま貼り付けられる
- キーを省くと【】内の語がキーになる
- 既にあるキーと重なるときは上書きせず、確認を求められる

@[...] は project-codes.md と照合され、一致した PJコードが工数表と週報に入る

場面	project-codes.md との関係
照合	@[...] の中身全体（/ を含むときは最後の / より後ろ。@[開発/クラウドデータ基盤] なら「クラウドデータ基盤」）を、表の1列目と完全一致で照らす
一致したとき	その行の PJコードを工数表のプロジェクトコード列に書き、週報ではプロジェクト名称も使う
同じコードの別キー	クラウドデータ基盤と DE勉強会はどちらも A313271100 で、週報の PJコード別集計では1つにまとまる
一致しないとき	推測でコードを書かずに空欄にし、記録の報告に「未登録: @[...]」と添える（@[未分類] は常に空欄）
業務終了時	空欄をもう一度照合して日中の登録分を反映し、残った未登録を報告する
週報	空欄の行を project-codes.md で補い、未登録のものは「(未登録)」として合計する
登録	「プロジェクトコードを追加: ...」で Excel の6列を貼り付け、キーを省くと【】内の語がキーになる

出典: project-codes.md、procedures/project-codes.md、procedures/day-end.md、AGENTS.md「週報の作成」

月末は「月末処理の工数入力データを作成して」の一言で、Excel にそのまま貼り付けるデータができる

準備と依頼

順番を書く	profile.md に、工数入力シートの行と同じ順番でプロジェクトIDを書いておく
依頼する	「月末処理の工数入力データを作成して（2026年10月）」と伝える（月を省くと尋ねられる）
貼り付ける	TSV を VS Code で開いて全選択・コピーし、1行目のプロジェクトの「1日」のセルに貼る

手順は procedures/monthly-hours.md（ファイルは scripts/month-hours.ps1 -Write だけが書く）

できるファイル（70_工数入力/）

2026/10.tsv	1行が1つのプロジェクトID、列は1日〜末日のタブ区切り
2026/10.md	名称・月間計・合計行と注記の付いた確認用の表

- 工数が0の ID も空欄の行で出るので、Excel の行とずれない
- 順番に無い ID の工数は入らず、注記と報告に出る
- TSV は手で直さず、日次ノートや profile.md を直して作り直す

週報は上司が2〜3分で読める形に集約され、成果・課題・来週の予定と工数が並ぶ

「今週の週報を作成して」で行われること

週を決める	get-week.ps1 で月曜から金曜と各日の日次ノートの有無を確かめる
テーマで整理	実績（目的・実施・成果・次のアクション）、課題・リスク、来週の予定、相談・共有事項に分ける
工数を合計する	カテゴリ別と PJコード別に、週の合計まで出す
書き出す	20_Weekly/ に下書きを作り、名前は profile.md の「自分の名前」から入れる

書き方の決まり（AGENTS.md）

- 結論を先に書き、「実施した」で終えずに確認・決定したことまで書く
- 会議への参加は実績にせず、決定事項を書く
- 記録に無い成果や数値は推測で書かない
- 載せたい成果は「内容:」「完了」に含め、載せない内容は「メモ:」で伝える

更新と保守

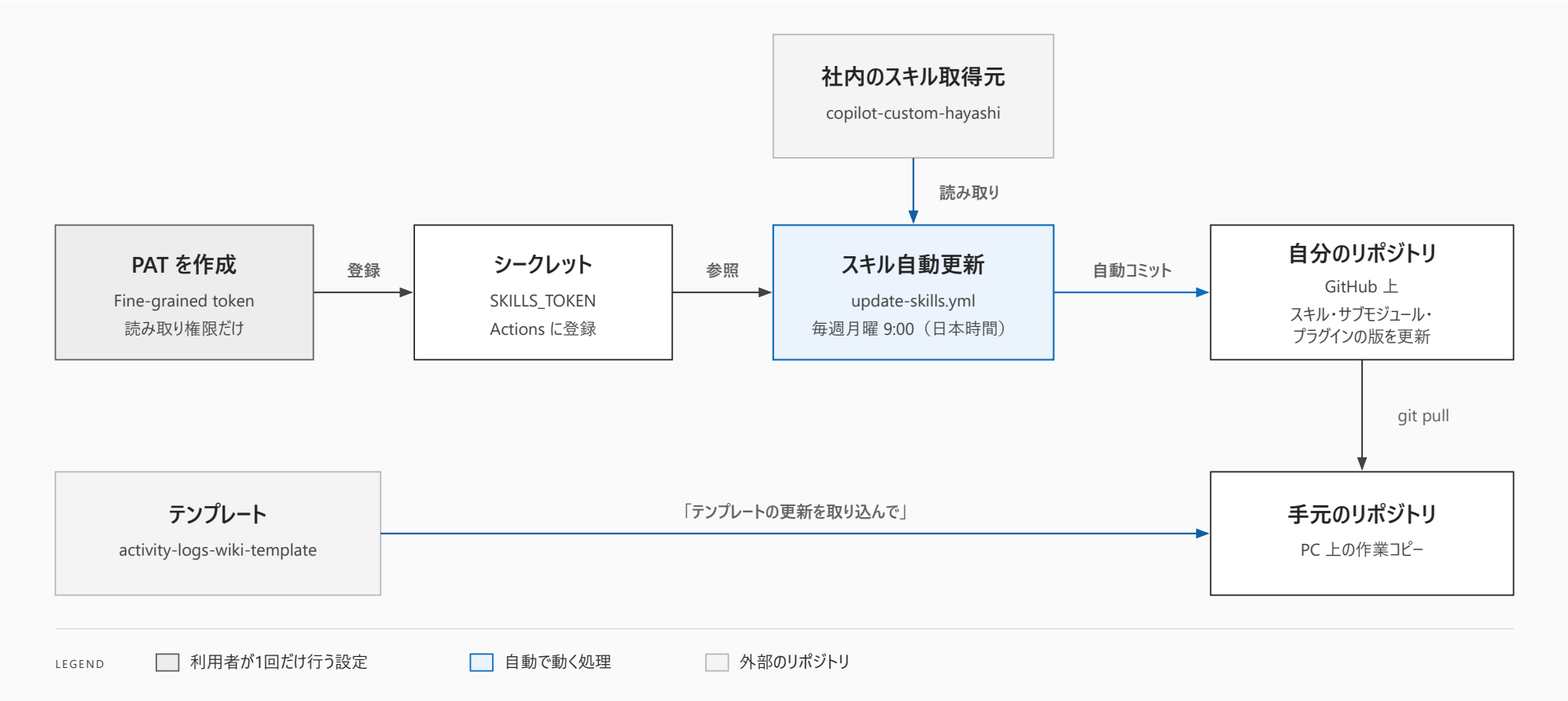
01 準備 P.2

02 スキルと wiki P.10

03 毎日の使い方 P.17

04 更新と保守 P.26

PAT を SKILLS_TOKEN に登録しておくと、スキルが毎週月曜に自動で更新される



PAT が要るのは、一部のスキルを社内リポジトリ copilot-custom-hayashi から取得するため

PAT は copilot-custom-hayashi の読み取りだけに絞って作り、SKILLS_TOKEN という名前で登録する

項目	設定値	補足
トークンの種類	Fine-grained token	Settings の Developer settings から作る
Resource owner	`ogis-dac-rd`	組織の承認が要る場合がある
Repository access	`ogis-dac-rd/copilot-custom-hayashi`	このリポジトリだけを選ぶ
Permissions	code と metadata を Read	画面では Contents と表示される場合がある（要確認）
登録先	自分のリポジトリの Actions シークレット	名前は `SKILLS_TOKEN`
動作確認	Actions の「Update Agent Skills and Plugins」を手動実行	Run workflow から実行する

テンプレートの更新は差分を確かめてから取り込み、記録と個人設定には触れない

区分	対象	扱い
取り込む	procedures/、scripts/、.agents/skills/、.claude/settings.json、.vscode/、.mcp.json、.github/hooks/ など	テンプレートの内容で更新する
要確認	AGENTS.md、CLAUDE.md、README.md、templates/、.github/（hooks/ を除く）	自分の変更を残して統合し、競合は判断を求められる
触らない	10_Daily/ などの記録、profile.md、project-codes.md、個人用の *.local.* ファイル	読み書きせず、変更があったことだけ報告する
退避する	テンプレートで削除されたファイル	削除せず 99_Archive/template-removed/ に移す
更新する	.gitmodules に書かれたサブモジュール	追跡ブランチの最新に更新する

「テンプレートの更新を取り込んで」と伝えると、差分の一覧で確認を取ってから反映し、取り込んだ版を .template-version に記録してコミットする

Appendix

トリガー早見表：チャットで使う言い方の一覧 P.29

資料のデザインシステム：Fluent 2 Web と作り方 P.30

トリガー早見表（複数に当てはまる言い方は、文言がいちばん近いものになる）

言い方	動作	手順・規則
本日の業務を開始する	日次ノートの作成と計画	`procedures/day-start.md`
<タスク>を開始／内容: ...／完了／中断	タスクの記録と工数の計上	`procedures/task-log.md`
メモ: ...／調査: ...／問題: ...／アイデア: ...	メモ・調査ノート・問題の記録	`procedures/task-log.md`
本日の業務を終了する	工数表の補完とコミット	`procedures/day-end.md`
プロジェクトコードを追加: ...	対応表に行を追加	`procedures/project-codes.md`
今週の週報を作成して	週報の作成	`AGENTS.md`
月末処理の工数入力データを作成して	工数入力ブック用のデータ作成	`procedures/monthly-hours.md`
wikiに追加: <URL/パス>／wikiで調べて／wikiを点検	wiki の取り込み・問い合わせ・点検	`llm-wiki`
ナレッジグラフを更新して	ナレッジグラフの作成（Claude Code）	`procedures/knowledge-graph.md`
テンプレートの更新を取り込んで	テンプレートの更新を反映	`procedures/template-update.md`

資料は Fluent 2 Web の色と書体でそろえ、スライドは角丸と影を使わない

要素	ガイド（HTML）	スライド（HTML・PDF・PPTX）
デザインシステム	Fluent 2 Web のトークンと部品	Fluent 2 Web の色と書体
色	ブランド #0f6cbd、文字 #242424、面 #fafafa、線 #d1d1d1	ブランド #0f6cbd、文字 #242424、面 #f5f5f5、線 #d1d1d1
書体	Segoe UI と Yu Gothic UI	Segoe UI と Yu Gothic（PPTX では Yu Gothic）
角丸・影	角丸 4px・8px と影を使う	設計規約に合わせて使わない
部品	Button・Input・Nav・表・MessageBar・Card	パーツ集の表・行のリスト・数字のブロック
図	diagram-design の fluent2-web で描き、青い塗りの上の文字だけ #111111 にする	

色の値は @fluentui/tokens（brandWeb・webLightTheme）による

同じ見た目の資料は fluent2-web とスキルで作り、HTML を直してから PPTX に変換する

同じ見た目で作る手順

図の設定	リポジトリの .diagram-design を profile: fluent2-web にする
作る	図は diagram-design、説明資料は design-taste-frontend、スライドはスライド作成スキルで作る
検査する	各スキルの検査スクリプトを回し、FAIL を 0 にする
直す	スライドは HTML を直してから PPTX に変換し直す

プロファイルは guides/profiles/ に同梱しており、図のスキルは ~/.diagram-design/profiles/ だけを読むので、別の PC ではそこへ写す

Fluent 2 から変えた点

- スライドは設計規約に合わせて角丸と影を使わない
- スライドの和文は字幅の広い Yu Gothic にする
- ガイドの本文は和文のため行送りを 1.75 に広げる
- 図の青い塗りの上の文字はコントラストのため #111111 にする

クラウドデータ基盤

詳しい手順は解説資料 2026-10-23_activity-logs-wiki-guide.html を参照

本資料は consulting-pptx-skill（github.com/carnot-tech/consulting-pptx-skill）で作成